

Data Collab RESTful API Overview

- [Authentication](#)
- [Assumption](#)
- [API Endpoints](#)
 - [SIT](#)
 - [UAT](#)
 - [PRD](#)
- [API Specification](#)
 - [Requests](#)
 - [POST](#)
 - [Create Request](#)
 - [Share Request](#)
 - [Group Request](#)
 - [Copy Request](#)
 - [PUT](#)
 - [Update Request](#)
 - [Update Request Status](#)
 - [Update Request Assignee](#)
 - [GET](#)
 - [Get Request](#)
 - [Get Requests](#)
 - [Get Request's shared users](#)
 - [Get Request's shared user groups](#)
 - [Get Request's view](#)
 - [Get Request's group](#)
 - [Get Request's amendment log \(beta\)](#)
 - [DELETE](#)
 - [Un-share Request](#)
 - [Un-group Request](#)
 - [Sheets](#)
 - [POST](#)
 - [Create Sheet](#)
 - [Add Sheet Image](#)
 - [Add Sheet Attachment](#)
 - [PATCH](#)
 - [Update Sheet \(partial update\)](#)
 - [PUT](#)
 - [Update Sheet \(total replacement\)](#)
 - [GET](#)
 - [Get Sheet](#)
 - [DELETE](#)
 - [Remove Sheet Image](#)
 - [Remove Sheet Attachment](#)
 - [Remove Sheet Rows](#)
 - [Images](#)
 - [GET](#)
 - [Get Image](#)
 - [Attachments](#)
 - [GET](#)

- [Get Attachment](#)
- [Collections](#)
 - [POST](#)
 - [Create Collection](#)
 - [PUT](#)
 - [Update Collection](#)
 - [GET](#)
 - [Get Collection](#)
 - [Get Collections](#)
 - [Get Collection sheet](#)
- [Documents](#)
 - [POST](#)
 - [Create Document](#)
 - [PUT](#)
 - [Update Document](#)
 - [PATCH](#)
 - [Update Document Name](#)
 - [GET](#)
 - [Get Document](#)
 - [Get Documents](#)
- [Views](#)
 - [POST](#)
 - [Create View](#)
 - [PUT](#)
 - [Update View](#)
 - [GET](#)
 - [Get View](#)
 - [Get Views](#)
- [Statuses](#)
 - [POST](#)
 - [Create Status](#)
 - [PUT](#)
 - [Update Status](#)
 - [GET](#)
 - [Get Status](#)
 - [Get Statuses](#)
- [Workspaces](#)
 - [POST](#)
 - [Create workspaces](#) (next phase)
 - [PUT](#)
 - [Update Workspaces](#) (next phase)
 - [GET](#)
 - [Get Workplace](#)
 - [POST](#)
 - [Add Workspace User](#)
 - [PUT](#)
 - [Update Workspace User's authorizations](#)
 - [GET](#)
 - [Get Workspace User](#)
 - [Get Workspace Users](#)
- [StatusCategories](#)
 - [GET](#)
 - [Get Status Categories](#)
- [Users](#)

- POST
 - Create User
- PUT
 - Update User (next phase)
- GET
 - Get User
 - Get Users
 - Get Me
- UserGroups
 - POST
 - Create User Group
 - PUT
 - Update User Group
 - GET
 - Get User Group
 - Get User Groups
- Date Time Format
 - Timestamp
- Status Code

Authentication

API Key is used which is a secure credential that is tied to user account. This key should be included in the header of the request for example as below:

```
1 | x-api-key:D99213612C3198C85CA2F4ABD62D2
```

new field for API key will be defined in MongoDB Users collection.

i.e. x-api-key: '17BE36E128F983C5CF2B23E4BC8C2'

Assumption

This API is intended and should be called by server-side application instead of user client directly. For security, It will result in Cross-Origin Resource Sharing (CORS) errors if using in a client-side application.

API Endpoints

The base URL under different environments as below:

SIT

<https://dtc-api-sit.lfuat.net/api>

UAT

<https://dtc-api.lfuat.net/api>

<https://dtc-api.lfapps.net/api>

API Specification

Requests

POST

Create Request

POST /v1/requests - *create a request record*

- mandatory checking for requestReference, requestDescription, documentId and workspaceId
- validation of documentId and workspaceId
- user authorization checking of workspaceId

e.g. /v1/requests

Request body:

```
1 {  
2   "requestReference" : "TW TESTING 20250103",  
3   "requestDescription" : "TW TESTING 20250103",  
4   "documentId" : "66d016d10a60b06ee18dd048",  
5   "workspaceId" : "66d7c3fc61bcd115bd56bf2f"  
6 }
```

Response code: 201

Response body:

```
1 {  
2   "data": {  
3     "requestId": "677794b0d5bc7818309e6dfe"  
4   }  
5 }
```

Share Request

POST /v1/requests/{requestId}/shares/{userId} - *share a request to a user by user id*

- validation (duplication, id existence) of {userId}
- user authorization checking of request
- message (optional): customized text in email content
- sendEmail (optional): email sending (Y / N), default is Y if not specified

e.g. /v1/requests/shares/670e275c2ac1b0026b5e55a9

Request body:

```

1 {
2   "viewNames": [
3     "View 1",
4     "View 2",
5     "View 3"
6   ],
7   "message" : "Hello",
8   "sendEmail" : "Y"
9 }

```

Response code: 201

POST /v1/requests/{requestId}/shares/{userEmail} - **share a request to a user by user email address**

- validation (duplication, existence) of {userEmail}
- user authorization checking of request
- message (optional): customized text in email content
- sendEmail (optional): email sending (Y / N), default is Y if not specified

e.g

e.g. /api/v1/requests/670e275c2ac1b0026b5e55a9/shares/tommywong%40lifung.com

Request body:

```

1 {
2   "viewNames": [
3     "View 1",
4     "View 2",
5     "View 3",
6   ],
7   "message" : "Hello",
8   "sendEmail" : "Y"
9 }

```

Response code: 201

POST /v1/requests/{requestId}/shares/usergroups/{userGroupId} - **share a request to a user group by user group id**

- validation (duplication, id existence) of {userGroupId}
- user authorization checking of request
- message (optional): customized text in email content
- sendEmail (optional): email sending (Y / N), default is Y if not specified

e.g. /v1/requests/shares/usergroups/670e275c2ac1b0026b5e55a9

Request body:

```

1 {
2   "viewNames": [
3     "View 1",

```

```

4         "View 2",
5         "View 3"
6     ],
7     "message" : ""
8 }

```

Response code: 201

POST /v1/requests/{requestId}/shares/usergroups/{userGroupName} - **share a request to a user group by user group name**

- validation (duplication, existence) of {userGroupName}
- user authorization checking of request
- message (optional): customized text in email content
- sendEmail (optional): email sending (Y / N), default is Y if not specified

e.g. /api/v1/requests/670e275c2ac1b0026b5e55a9/shares/usergroups/testing%20group

Request body:

```

1 {
2     "viewNames": [
3         "View 1",
4         "View 2",
5         "View 3",
6     ]
7 }

```

Response code: 201

Group Request

POST /v1/requests/{requestId}/groups/{requestIdToBeGrouped} - **group Request**

- check id existence of {requestId} and {requestIdToBeGrouped} from collection Requests
- {requestIdToBeGrouped} cannot be existed in collection RequestGroups
- user authorization

e.g. /v1/requests/670e275c2ac1b0026b5e55a9/groups/35464ff1b0026b5e55a9

Response code: 201

Copy Request

POST /v1/requests/{requestId}/copy - **copy a request and sheet data**

- `copySheet` : "Y" - include sheet data

e.g. /v1/requests/670e275c2ac1b0026b5e55a9/copy

Request body:

```

1 {
2   "requestReference" : "TW TESTING 20250103",
3   "requestDescription" : "TW TESTING 20250103",
4   "documentId" : "66d016d10a60b06ee18dd048",
5   "copySheet": "Y"
6 }

```

Response code: 201

PUT

Update Request

PUT /v1/requests/{requestId} - **update a request record**

- mandatory checking for requestReference and requestDescription
- validation of collectionId (id existence if any)

e.g. /v1/requests/677794b0d5bc7818309e6dfe

Request body:

```

1 {
2   "requestReference" : "TW TESTING 20250103",
3   "requestDescription" : "TW TESTING 20250103",
4   "collectionId" : "677de826c491e9c07833f862"
5 }

```

Response code: 204

Update Request Status

PUT /v1/requests/{requestId}/statuses/{statusId} - **update status by status ID**

- validation (id existence)

e.g. /v1/requests/677794b0d5bc7818309e6dfe/statuses/669745da8c01029542e88592

Response code: 200

PUT /v1/requests/{requestId}/statuses/{statusName} - **update status by status name**

- validation of requestId and statusName
- return error if more than one Status ID found
- user authorization

e.g. /v1/requests/677794b0d5bc7818309e6dfe/statuses/Done

Response code: 200

Update Request Assignee

PUT /v1/requests/{requestId}/assignees/{assigneeUserId} - **update assignee by User ID**

- validation of requestId and assigneeUserId

- user authorization
- ccEmailToAssigner (optional): cc email to Assigner (follow user preference by default)

e.g. `/v1/requests/677794b0d5bc7818309e6dfe/assignees/670e275c2ac1b0026b5e55a9`

Request body:

```
1 {
2   "ccEmailToAssigner" : "N"
3 }
```

Response code: 200

PUT `/v1/requests/{requestId}/assignees/{assigneeEmail}` - **update assignee by email address**

- validation
- user authorization
- ccEmailToAssigner (optional): Send email to Assigner (follow user preference by default)

e.g. `/v1/requests/677794b0d5bc7818309e6dfe/assignees/tommywong%40lifung.com`

Request body:

```
1 {
2   "ccEmailToAssigner" : "N"
3 }
```

Response code: 200

GET

Get Request

GET `/v1/requests/{requestId}` - **get a single request record**

- return active record only

e.g. `/v1/requests/677794b0d5bc7818309e6dfe`

GET `/v1/requests?requestreference={requestReference}&workspacename={workspace}`
- **get a single request record by request ref. under workspace**

- exact match
- return error if more than one record returned
- return active record only

e.g. `/v1/requests?requestreference=TW20240813&workspacename=workspace`

GET /v1/requests - **search and list request record(s) with criteria**

- `workspaceName` - workspace
- `pendingOnly` : (Y/N) - filter with pending records only
- `requestOnly` : (Y/N) - filter with my request records only
- `filters` - filtering criteria - AND followings:
(matching by contain)
 - `requestReference` - Request Reference
 - `requestDescription` - Request Description
 - `documentName` - Document Name
 - `collectionName` - Collection Name
 - `ownerEmail` - Owner's email address
 - `assigneeEmail` - Assignee's email address
 - `requestStatusName` - Request's status
 - `requestIsActive` - Request Active or not (return all records if not specified)

e.g. /v1/requests

Request body:

```
1 {
2   "pendingOnly": "Y",
3   "requestOnly": "N",
4   "workspaceName": "workspace",
5   "filters": {
6     "requestReference": "TW",
7     "requestDescription": "TW",
8     "documentName": "Doc Type 3",
9     "collectionName": "TW20240822",
10    "ownerEmail": "tommywong@lifung.com",
11    "assigneeEmail": "v-desmondchan@lifung.com",
12    "requestStatusName": "Status 2 (3)",
13    "requestIsActive": "Y"
14  }
15 }
```

Response code: 200

Response body:

```
1 {
2   "data": [
3     {
4       "requestId": "666fce3f96d13db46ab74941",
5       "requestDescription": "Testing",
6       "requestReference": "TW TEST 20240617",
7       "requestStatusId": "66949f005b401addca7de05",
8       "requestStatusName": "Working Recap",
9       "requestIsActive": "Y",
10      "sheetId": "66949f005434343addca7de05",

```

```

11         "statusCategoryData": {
12             "statusCategoryId": "670dc553d3bc14b6e8c284ac",
13             "statusCategoryName": "To Do",
14             "backgroundColor": "#F1F2F4",
15             "textColor": "#172B4D",
16             "isDefault": "Y",
17             "isCompleted": "N"
18         },
19         "documentId": "66865e9e69c23b8350b41f02",
20         "documentName": "Quotation 44",
21         "assigneeId": "66ab34ac9177479a52954eb",
22         "assigneeName": "Alvin Teng",
23         "assigneeUserType": "LF",
24         "categoryId": null,
25         "collectionId": null,
26         "collectionName": "",
27         "ownerId": "670e275c2ac1b002wb5e55a9",
28         "ownerName": "Tommy Wong",
29         "ownerUserType": "LF",
30         "workspaceId": "66d7c3fc61bcd115bd56bf2f",
31         "createdBy": "670e275c2ac1b0d26b5e55a9",
32         "createdByUserName": "Tommy Wong",
33         "crreatedDat": "2025-01-08T02:51:18.171Z",
34         "updatedBy": "670e275c2ac1b00d6b5e55a9",
35         "updatedByUserName": "Tommy Wong",
36         "updatedDat": "2025-01-08T02:51:18.171Z",
37     },
38     {
39         "requestId": "66bb2aa1d1059bf657ea5523",
40         "requestDescription": "TW20240813",
41         "requestReference": "TW20240813",
42         "requestStatusId": "6694d3745b401adddca7de3e",
43         "requestStatusName": "Status 2 (3)",
44         "requestIsActive": "Y",
45         "statusCategoryData": {
46             "id": "670dc553d3bc14b6e8c284ac",
47             "name": "To Do",
48             "backgroundColor": "#F1F2F4",
49             "textColor": "#172B4D",
50             "isDefault": "Y",
51             "isCompleted": "N"
52         },
53         "assigneeId": "",
54         "assigneeName": "Not Assigned",
55         "assigneeUserType": "",
56         "categoryId": null,
57         "collectionId": "66c6aeec4b033fa171d3f9d4",
58         "collectionName": "TW20240822",
59         "docTypeId": "668f90e15b401adddca7ddce",
60         "docTypeName": "Doc Type 3",
61         "ownerId": "670e275c2ac1b0026b5e55a9",
62         "ownerName": "Tommy Wong",
63         "ownerUserType": "LF",
64         "workspaceId": "66d7c3fc61bcd115bd56bf2f",
65         "createdBy": "670e275c2ac1b0026b5e55a9",
66         "createdByUserName": "Tommy Wong",
67         "createdDat": "2025-01-08T02:51:18.171Z",
68         "updatedBy": "670e275c2ac1b0026b5e55a9",
69         "updatedByUserName": "Tommy Wong",
70         "updatedDat": "2024-12-18T06:02:27.034Z",
71     }
72 ]
73 }

```

Get Request's shared users

GET /v1/requests/{requestId}/shares - **list shared users of a single request record**

Response code: 200

Response body:

```
1 {
2   "data": [
3     {
4       "userId": "667b86b3b19f047cbbbb8d94",
5       "userName": "Manh Tran",
6       "userEmail": "manh@lifung.com"
7     },
8     {
9       "userId": "673c63e42ac1b0026b5e55bb",
10      "userName": "Desmond Chan",
11      "userEmail": "desmond@lifung.com"
12    }
13  ]
14 }
```

Get Request's shared user groups

GET /v1/requests/{requestId}/shares/usergroups - **list shared user groups of a single request record**

Response code: 200

Response body:

```
1 {
2   "data": [
3     {
4       "userGroupId": "667b86b3b19f047cbbbb8d94",
5       "userGroupName": "User Group 1",
6       "users": [
7         {
8           "userId": "673c63e42ac1b0026b5e55bb",
9           "userName": "Desmond Chan",
10          "userEmail": "desmond@lifung.com"
11        },
12        {
13          "userId": "673c63e42ac1b0026b5edddd",
14          "userName": "Tommy Wong",
15          "userEmail": "tommywong@lifung.com"
16        }
17      ]
18    },
19    {
20      "userGroupId": "673c63e42ac1b0026b5e55bb",
21      "userGroupName": "User Group 2",
22      "users": [
23        {
24          "userId": "667b86b3b19f047cbbbb8d94",
25          "userName": "Manh Tran",
26          "userEmail": "manh@lifung.com"
27        }
28      ]
29    }
30  ]
31 }
```

Get Request's view

GET /v1/requests/{requestId}/views - **list available views of a single request record**

Response code: 200

Response body:

```
1 {
2   "data": [
3     {
4       "viewId": "67a087b0ac6897d6161e8206",
5       "viewName": "View 1"
6     },
7     {
8       "viewId": "67a08a39ac6897d6161e8209",
9       "viewName": "View 2"
10    }
11  ]
12 }
```

Get Request's group

GET /v1/requests/{requestId}/groups - **list grouped requests**

Response code: 200

Response body:

```
1 {
2   "data": [
3     {
4       requestId: '66b33a3b8d9591a04edd21b8'
5     },
6     {
7       requestId: '658bc114025900006b002c02'
8     },
9     {
10      requestId: '658bc114025900006b002c03'
11    },
12    {
13      requestId: '658bc114025900006b002c04'
14    }
15  ]
16 }
```

Get Request's amendment log (beta)

GET /v1/requests/{requestId}/amendmentlogs - **list amendment logs of a single request record**

viewId={string} - (mandatory except workspace admin) filter fields under the specified view

logDatStart={string} - (optional) filter logs from this date (ISO 8601, UTC)

logDatEnd={string} - (optional) filter logs up to this date (ISO 8601, UTC)

logByEmail={string} - (optional) filter by the user who made changes (exact match)

Request body:

e.g.

```

1 {
2   "filters": {
3     "viewId" : "694268ebba42f1789c5a9e9b"
4     "logDatStart": "2026-03-20T06:24:45.283Z",
5     "logDatEnd": "2026-03-31T06:24:45.283Z",
6     "action": "New",
7     "logByEmail": "tommywong@lifung.com"
8   }
9 }

```

Response code: 200

Response body:

Log records will be in descending order

e.g.

```

1 {
2   "data": [
3     {
4       "logId": "67ea352d6888c6dde7254559",
5       "logDat": "2025-03-31T06:24:45.283Z",
6       "logBy": "66ab34acf9177479a52954eb",
7       "logByEmail": "tommywong@lifung.com",
8       "logByUserName": "Tommy Wong",
9       "logDetails": [
10        {
11          "rowId": "c73320d4-6dcb-4030-b4f5-b9790f2829a6",
12          "action": "New"
13        },
14        {
15          "rowId": "c73320d4-6dcb-4030-b4f5-b9790f2829a6",
16          "fieldId": "66b5b6881137c20f55db3b76",
17          "fieldName": "supplier code",
18          "fieldType": "string",
19          "chgFrom": "",
20          "chgTo": "4",
21          "action": "Edit"
22        },
23        {
24          "rowId": "33377870-c465-4083-9707-5172aae7b907",
25          "fieldId": "6926aea2aa20c46b891efb15",
26          "fieldName": "Text Field",
27          "fieldType": "string",
28          "chgFrom": "Line 1\r\nLine 2\r\nLine 3\r\nLine 4\r\nLine
29          5\r\nLine 6\r\nLine 7\r\nLine 8\r\nLine 9",
30          "chgTo": "Line 1\r\nLine 2\r\nLine 3\r\nLine 4\r\nLine
31          6\r\nLine 7\r\nLine 8\r\nLine 9",
32          "action": "Edit"
33        },
34        {
35          "rowId": "6301a708-5d10-4eab-a01f-4d40049cf38e",
36          "fieldId": "6926aea2aa20c46b891efb15",
37          "fieldName": "Text Field",
38          "fieldType": "string",
39          "chgFrom": "",
40          "chgTo": "Line 1\r\nLine 2\r\nLine 3\r\nLine 4\r\nLine
41          5\r\nLine 6\r\nLine 7\r\nLine 8\r\nLine 9",
42          "action": "Edit"
43        }
44      ]
45    },
46    {
47      "logId": "69c0daaa70f463e6c8d8ff50",

```

```

45     "logDat": "2026-03-23T06:16:06.459Z",
46     "logBy": "6668157a372c7c4a9c182319",
47     "logByEmail": "tommywong@lifung.com",
48     "logByUsername": "Tommy Wong",
49     "logDetails": [
50       {
51         "rowId": "bulk",
52         "rowNumber": "bulk",
53         "changeCount": 92,
54         "action": "Edit"
55       }
56     ]
57   }
58 ]
59 }

```

DELETE

Un-share Request

DELETE /v1/requests/{requestId}/shares/{userId} - **un-share request of a user by User ID**

e.g. /v1/requests/677794b0d5bc7818309e6dfe/shares/673c63e42ac1b0026b5e55bb

Response code: 204

DELETE /v1/requests/{requestId}/shares/{userEmail} - **un-share request of a user by email address**

e.g. /v1/requests/677794b0d5bc7818309e6dfe/shares/tommywong%40lifung.com

Response code: 204

DELETE /v1/requests/{requestId}/shares/usergroups/{userGroupId} - **un-share request of a user group by user group ID**

e.g.

/v1/requests/677794b0d5bc7818309e6dfe/shares/usergroups/673c63e42ac1b0026b5e55bb

Response code: 204

DELETE /v1/requests/{requestId}/shares/usergroups/{userGroupName} - **un-share request of a user group by user group name**

e.g. /v1/requests/677794b0d5bc7818309e6dfe/shares/usergroups/testing%20group

Response code: 204

Un-group Request

DELETE /v1/requests/{requestId}/groups/{requestIdToBeUngrouped} - **un-group request**

- check id existence of {requestId} and {requestIdToBeUngrouped} from collection RequestGroups

e.g. /v1/requests/677794b0d5bc7818309e6dfe/groups/673c63e42ac1b0026b5e55bb

Response code: 204

Sheets

POST

Create Sheet

POST /v1/sheets - **create a request record and a sheet record**

- Workspace Name, View Name and Document Name will be used in JSON
- View validation - column exists, mandatory, readOnly
- Check user have view premission
- Request will be shared to `requestAssigneeEmail` as well
- `requestAssigneeCcEmailToAssigner` (Y/N) : whether system cc the email to assigner (follow user preference by default)
- `requestAssigneeSharingViewNames` is list of views access for the shared user `requestAssigneeEmail`
- view control validation (i.e. field mandatory and editable)

e.g. /v1/sheets

Request body:

```
1 {
2   "requestReference": "Ref Fabric - Desmond",
3   "requestDescription": "Ref Desc Fabric - Desmond",
4   "requestAssigneeEmail": "v-desmondchan@lifung.com",
5   "requestAssigneeSharingViewNames" : "View 1",
6   "requestAssigneeCcEmailToAssigner" : "N",
7   "requestOwnerUserName": "Tommy Wong",
8   "requestOwnerEmail": "tommywong@lifung.com",
9   "requestStatusName": "New",
10  "documentName": "Fabric Capture (Denim)",
11  "workspaceName": "workspace",
12  "viewName": "View 1",
13  "sheetData": [
14    {
15      "rowIndex": 1,
16      "Mill Article#": "111",
17      "Is Fabric Exclusive for Customer": "Yes",
18      "Coloration Process": "Cross Dyed",
19    },
20    {
21      "rowIndex": 2,
22      "Mill Article#": "222"
23    },
24    {
25      "rowIndex": 3,
26      "Mill Article#": "333",
27      "Is Fabric Exclusive for Customer": "No"
28    },
29    {
30      "rowIndex": 4,
```

```

31         "Mill Article#": "def",
32         "Coloration Process": "Fiber Heather) Dyed"
33     },
34     {
35         "rowIndex": 6,
36         "Mill Article#": "zxc",
37         "Is Fabric Exclusive for Customer": "Yes",
38         "Coloration Process": "Print"
39     }
40 ]
41 }

```

Response code: 201

Response body:

```

1 {
2   "data": {
3     "requestId": "677794b0d5bc7818309e6dfe",
4     "sheetId": "67468bc66d9bb8104c1bc6a6",
5   }
6 }

```

Add Sheet Image

POST /v1/sheets/{sheetId}/views/{viewId}/images?rowindex={number}&columnname={text}

- **add image to a sheet**

Max file size: 10Mb

Parameters:

rowindex - Row index in the sheet

columnname - Column name in the sheet

- mandatory checking for rowindex and columnname
- validation of rowindex and columnname
- system will upload image to Azure blob storage and update collection Sheets
- view control validation (i.e. field editable)

e.g.

/v1/requests/sheets/67468bc66d9bb8104c1bc6a6/views/67b2baf3b246932af071118d/images?
rowindex=1&columnname=Photo

multipart/form-data

Response code: 201

Response body:

```

1 {
2   "data": {
3     "fileName": "3264766d-8535-4ec2-af32-a43bce40eed6.jpg",
4     "thumbnailName": "3264766d-8535-4ec2-af32-a43bce40eed6-tn1.jpg",
5     "mimeType": "image/jpeg",

```



```

6         "originalName": "Flower_jtca001-a2f4.jpg",
7         "containerName": "xtsdtcnpsit"
8     }
9 }

```

Response code: 204

Add Sheet Attachment

POST /v1/sheets/{sheetId}/views/{viewId}/attachments?rowindex={number}&columnname={text}

- add attachment to a sheet

Max file size: 50Mb

Parameters:

rowindex - Row index in the sheet

columnname - Column name in the sheet

- mandatory checking for rowindex and columnname
- validation of rowindex and columnname
- system will upload attachment to Azure blob storage and update collection Sheets
- view control validation (i.e. field editable)
- supported file extension: PDF, JPG, JPEG, GIF, TIF, TIFF, XLS, XLSX, DOC, DOCX, TXT, MSG, ZIP

e.g.

/v1/requests/sheets/67468bc66d9bb8104c1bc6a6/views/67b2baf3b246932af071118d/attachments?rowindex=1&columnname=Attachment

multipart/form-data

Response code: 201

Response body:

```

1 {
2   "data": {
3     "mimeType": "application/pdf",
4     "fileName": "081fbf2e-1f8b-40d5-ab2a-e7b925df9896.pdf",
5     "originalName": "xxxxxx.pdf",
6     "containerName": "xtsdtcnpsit",
7     "thumbnailName": "081fbf2e-1f8b-40d5-ab2a-e7b925df9896-tn1.pdf"
8   }
9 }

```

Response code: 204

PATCH

Update Sheet (partial update)

PATCH /v1/sheets/{sheetId}/views/{viewId} - **partial update of sheet record**

- view control validation (i.e. field editable)
- locate row by rowIndex
- if rowId is specified, rowId will be used to locate row instead of rowIndex

e.g. /v1/requests/sheets/67468bc66d9bb8104c1bc6a6/views/67b2baf3b246932af071118d

Request body:

```
1 {
2   "sheetData": [
3     {
4       "rowIndex": 1,
5       "Mill Article#": "111",
6       "Is Fabric Exclusive for Customer": "Yes",
7       "Coloration Process": "Cross Dyed",
8     },
9     {
10      "rowIndex": 2,
11      "Mill Article#": "222"
12    },
13    {
14      "rowIndex": 3,
15      "Mill Article#": "333",
16      "Is Fabric Exclusive for Customer": "No"
17    },
18    {
19      "rowIndex": "dd6eb643-e000-4482-a279-c0c55cafb36a",
20      "Mill Article#": "def",
21      "Coloration Process": "Fiber Heather) Dyed"
22    },
23    {
24      "rowId": 6,
25      "Mill Article#": "zxc",
26      "Is Fabric Exclusive for Customer": "Yes",
27      "Coloration Process": "Print"
28    }
29  ]
30 }
```

Response code: 204

PUT

Update Sheet (total replacement)

PUT /v1/sheets/{sheetId}/views/{viewId} - **update sheet record (total replacement)**

*** please use PATCH Update Sheet (partial update) if documents contains formula, lookup, image and attachment field*

- view control validation (i.e. field editable)

- the view must include all fields of corresponding document

e.g. `/v1/requests/sheets/67468bc66d9bb8104c1bc6a6/views/67b2baf3b246932af071118d`

Request body:

```

1  {
2    "sheetData": [
3      {
4        "rowIndex": 1,
5        "Mill Article#": "111",
6        "Is Fabric Exclusive for Customer": "Yes",
7        "Coloration Process": "Cross Dyed",
8      },
9      {
10       "rowIndex": 2,
11       "Mill Article#": "222"
12     },
13     {
14       "rowIndex": 3,
15       "Mill Article#": "333",
16       "Is Fabric Exclusive for Customer": "No"
17     },
18     {
19       "rowIndex": 4,
20       "Mill Article#": "def",
21       "Coloration Process": "Fiber Heather) Dyed"
22     },
23     {
24       "rowIndex": 6,
25       "Mill Article#": "zxc",
26       "Is Fabric Exclusive for Customer": "Yes",
27       "Coloration Process": "Print"
28     }
29   ]
30 }

```

Response code: 204

GET

Get Sheet

GET `/v1/sheets/{sheetId}/views/{viewId}` - **get a single sheet record**

e.g. `/v1/sheets/677f71ab2c7197bd8203eb06/views/677f71402c7197bd8203eb03`

filters:

- retrieve a subset of data based on specified criteria
- AND will be used between the field
- Date format will follow the user preference

operator

- **Equality:** (case-insensitive) `=` (equal to), `!=` (not equal to)

- **Comparison:** (*case-sensitive*) `>` (greater than), `<` (less than), `>=` (greater than or equal to), `<=` (less than or equal to)
- **Containment:** (*case-insensitive*) `~` (contains), `!~` (does not contain), `in` (included in an array), `not in` (not included in an array)

Available operators for field type:

- Text: equality, comparison, containment
- Number: equality, comparison, containment
- Date: equality, comparison, containment(`in`, `not in`)
- Checkbox: (*YES, NO, Empty String*) equality, containment(`in`, `not in`)
- Dropdown list: equality, comparison (*compare by string only*), containment
- Dynamic Dropdown: equality, comparison (*compare by string only*), containment
- Multiple Dropdown list: containment(`in`, `not in`)
- URL: equality, comparison, containment
- LOOKUP: equality, comparison, containment
- Attachment: (*meta data*) equality, comparison, containment
- Image: (*meta data*) equality, comparison, containment

Request Body:

```

1  {
2      "filters": {
3          "TextField": {
4              "operator" : "=",
5              "values": "TW"
6          },
7          "TextField2": {
8              "operator" : "!=",
9              "values": ""
10         },
11         "NumerField": {
12             "operator" : ">",
13             "values": 10
14         },
15         "TextField3": {
16             "operator" : "~",
17             "values": "TW"
18         },
19         "TextField4": {
20             "operator" : "in",
21             "values": ["A", "B", "C"]
22         }
23     }
24 }

```

Response Code: 200

Response Body:

```

1  {
2      "sheetId": "67468bc66d9bb8104c1bc6a6",
3      "requestReference": "Ref Fabric - Desmond",
4      "requestDescription": "Ref Desc Fabric - Desmond",
5      "requestAssigneeUserName": "Desmond Chan",
6      "requestAssigneeEmail": "v-desmondchan@lifung.com",
7      "requestOwnerUserName": "Desmond Chan",
8      "requestOwnerEmail": "v-desmondchan@lifung.com",
9      "requestStatus": "New",
10     "documentName": "Fabric Capture (Denim)",
11     "workspaceName": "workspace",
12     "sheetData": [
13         {
14             "rowId": "dd6eb643-e000-4482-a279-c0c55cafb36a",
15             "rowIndex": 1,
16             "Mill Article#": "111",
17             "Is Fabric Exclusive for Customer": "Yes",
18             "Coloration Process": "Cross Dyed",
19             "Fabric Image": "https://dtd-api-
sit2.lfuat.net/uploads/6bfc7a27-53f7-4543-b6de-d15d8ab1f0be-tn1.png"
20         },
21         {
22             "rowId": "ce3c8c62-97d9-412d-b57a-1c6df9f06f1f",
23             "rowIndex": 2,
24             "Mill Article#": "222"
25         },
26         {
27             "rowId": "c6ffa018-c27c-4fa9-89e7-886e5dd16dce",
28             "rowIndex": 3,
29             "Mill Article#": "333",
30             "Is Fabric Exclusive for Customer": "No"
31         },
32         {
33             "rowId": "d6793b02-8410-464c-9d7a-686d395c3850",
34             "rowIndex": 4,
35             "Mill Article#": "def",
36             "Coloration Process": "Fiber Heather) Dyed"
37         },
38         {
39             "rowId": "13808548-000b-45fa-8574-75d5b79f3ddf",
40             "rowIndex": 6,
41             "Mill Article#": "zxc",
42             "Is Fabric Exclusive for Customer": "Yes",
43             "Coloration Process": "Print"
44         }
45     ],
46     "createdByUserId": "670e275c2ac1b0026b5e55a9",
47     "createdByUserName": "Tommy Wong",
48     "createdDate": "2025-01-08T02:51:18.171Z",
49     "updatedByUserId": "670e275c2ac1b0026b5e55a9",
50     "updatedByUserName": "Tommy Wong",
51     "updatedAt": "2025-01-08T02:51:18.171Z"
52 }

```

DELETE

Remove Sheet Image

DELETE /v1/sheets/{sheetId}/views/{viewId}/images?rowindex=1&columnname=Photo

- delete image from a sheet

Parameters:

rowindex - Row index in the sheet

columnname- Column name in the sheet

- mandatory checking for rowindex
- validation of rowindex and columnname
- view control validation (i.e. field editable)

e.g.

`/v1/requests/sheets/67468bc66d9bb8104c1bc6a6/views/67b2baf3b246932af071118d/images?
rowindex=1&columnname=Photo`

Response code: 204

Remove Sheet Attachment

DELETE `/v1/sheets/{sheetId}/views/{viewId}/attachments?
rowindex=1&columnname=Attachment`

- delete attachment from a sheet

Parameters:

rowindex - Row index in the sheet

columnname- Column name in the sheet

- mandatory checking for rowindex
- validation of rowindex and columnname
- view control validation (i.e. field editable)

e.g.

`/v1/requests/sheets/67468bc66d9bb8104c1bc6a6/views/67b2baf3b246932af071118d/attachm
ents?rowindex=1&columnname=Photo`

Response code: 204

Remove Sheet Rows

DELETE `/v1/sheets/{sheetId}/views/{viewId}/rows - delete multiple rows from a sheet`

e.g. `/v1/sheets/67468bc66d9bb8104c1bc6a6/views/67b2baf3b246932af071118d/rows`

Request body:

```
1 {  
2   "rowIndexes": [1,2,5,6]  
3 }
```

- validation - user permission

- view control validation

e.g.

* Document

* Name

* Order Allow Insert Row ☒ Allow Remove Row ☒

- update row indexes of the sheet

Response code: 204

Images

GET

Get Image

GET /v1/images/{filename] - **get a image**

e.g. /v1/images/eb1a0bb7-31f6-4455-aa4d-3105f2092ace.png

Response code: 200

Attachments

GET

Get Attachment

GET /v1/attachments/{filename] - **get an attachment**

e.g. /v1/attachments/e0f401f7-720b-4071-9c93-e2ee1e917cbf.pdf

Response code: 200

Collections

POST

Create Collection

POST /v1/collections - **create a collection record**

- mandatory checking for `collectionName`, `documentId`, `viewId`, `workspaceId`
- validation of `requestId` and `workspaceId`

e.g. /v1/collections

Request body:

```

1  {
2      "collectionName" : "TW TESTING COLLECTION",
3      "documentId" : "66d016d10a60b06ee18dd048",
4      "viewId" : "66d019370a60b06ee18dd054",
5      "workspaceId" : "66d7c3fc61bcd115bd56bf2f",
6      "requests": [
7          {
8              "requestId": "677794b0d5bc7818309e6dfe"
9          }
10     ]
11 }

```

Response code: 201

Response body:

```

1  {
2      "data": {
3          "collectionId": "677de826c491e9c07833f862"
4      }
5  }

```

PUT

Update Collection

PUT /v1/collections/{collectionId} - **update collection record**

- mandatory checking for collectionName, documentId, viewId
- validation of requestId

e.g. /v1/collections/677de826c491e9c07833f862

Request body:

```

1  {
2      "collectionName" : "TW TESTING COLLECTION",
3      "documentId" : "66d016d10a60b06ee18dd048",
4      "viewId" : "66d019370a60b06ee18dd054",
5      "requests": [
6          {
7              "requestId": "677794b0d5bc7818309e6dfe"
8          }
9      ]
10 }

```

Response code: 204

GET

Get Collection

GET /v1/collections/{collectionId} - **get a single collection record**

e.g. /v1/collections/677de826c491e9c07833f862

Get Collections

GET /v1/collections - **search and list record(s) with criteria**

Request parameter:

~~page={number}~~

~~pagesize={number}~~

workspacename={string} (whole word match)

collectionname={string} (partial word match)

e.g. /v1/collections?~~page=1&pagesize=25&keyword=TW~~

e.g. /v1/collections?workspacename=workspace&collectionname=TW

Response code: 200

Response body:

```
1 {
2   "data": [
3     {
4       "collectionId": "677de826c491e9c07833f862",
5       "collectionName": "TW TESTING COLLECTION",
6       "documentId": "66d016d10a60b06ee18dd048",
7       "documentName": "Quotation 2.",
8       "dynamicFields": null,
9       "requests": [
10        {
11          "requestId": "677794b0d5bc7818309e6dfe",
12          "requestRef": "TW TESTING 20250103",
13          "requestDesc": "TW TESTING 20250103"
14        }
15      ],
16      "viewId": "66d019370a60b06ee18dd054",
17      "viewName": "Order Recap",
18      "workspaceId": "66d7c3fc61bcd115bd56bf2f",
19      "createdByUserId": "670e275c2ac1b0026b5e55a9",
20      "createdByUserName": "Tommy Wong",
21      "createdDate": "2025-01-08T02:51:18.171Z",
22      "updatedByUserId": "670e275c2ac1b0026b5e55a9",
23      "updatedByUserName": "Tommy Wong",
24      "updatedAt": "2025-01-08T02:51:18.171Z"
25    },
26    {
27      "collectionId": "677de826c491e9c07833f863",
28      "name": "TW TESTING COLLECTION 2",
29      "documentId": "66d016d10a60b06ee18dd048",
30      "documentName": "Quotation 2.",
31      "dynamicFields": null,
32      "requests": [
33        {
34          "requestId": "677794b0d5bc7818309e6dfe",
35          "requestReference": "TW TESTING 20250103",
36          "requestDescription": "TW TESTING 20250103"
37        }
38      ],
39      "viewId": "66d019370a60b06ee18dd054",
40      "viewName": "Order Recap",
41      "workspaceId": "66d7c3fc61bcd115bd56bf2f",
42      "createdByUserId": "670e275c2ac1b0026b5e55a9",
43      "createdByUserName": "Tommy Wong",
44      "createdDate": "2025-01-08T02:51:18.171Z",
45      "updatedByUserId": "670e275c2ac1b0026b5e55a9",
```

```

46         "updatedByUsername": "Tommy Wong",
47         "updatedAt": "2025-01-08T02:51:18.171Z"
48     },
49 ],
50     "pagination" : {
51         "totalRecord" : 100,
52         "currentPage" : 1,
53         "totalPage" : 10,
54         "nextPage" : 2,
55         "previousPage" : null
56     }
57 }

```

Get Collection sheet

GET /v1/collections/{collectionId}/sheets - **get a single collection sheet record**

Request Parameter

viewid = {string} optional

*** follow defined view if viewid not specified*

e.g. /v1/requests/677de826c491e9c07833f862/sheets

e.g. /v1/requests/677de826c491e9c07833f862/sheets?viewid=668f9fd2799ab11b7c41b09f

```

1  {
2      "collectionId": "677de826c491e9c07833f862",
3      "collectionName": "TW20240822",
4      "documentName": "Fabric Capture (Denim)",
5      "viewId": "668f9fd2799ab11b7c41b09f",
6      "viewName": "Veiw 1 (DocType3)",
7      "requests": [
8          {
9              "requestId": "66bb2aa1d1059bf657ea5523",
10             "requestReference": "TW20240813",
11             "requestDescription": "TW20240813"
12         },
13         {
14             "requestId": "66b042cf94b62c6f7170e863",
15             "requestRef": "Testing1234"
16         }
17     ],
18     "workspaceId": "66d7c3fc61bcd115bd56bf2f",
19     "workspaceName": "workspace",
20     "sheetData": [
21         {
22             "rowId": "dd6eb643-e000-4482-a279-c0c55cafb36a",
23             "rowIndex": 1,
24             "Mill Article#": "111",
25             "Is Fabric Exclusive for Customer": "Yes",
26             "Coloration Process": "Cross Dyed",
27             "Fabric Image": "https://dtc-api-
sit2.1fuat.net/uploads/6bfc7a27-53f7-4543-b6de-d15d8ab1f0be-tn1.png"
28         },
29         {
30             "rowId": "ce3c8c62-97d9-412d-b57a-1c6df9f06f1f",
31             "rowIndex": 2,
32             "Mill Article#": "222"
33         },
34         {
35             "rowId": "c6ffa018-c27c-4fa9-89e7-886e5dd16dce",
36             "rowIndex": 3,
37             "Mill Article#": "333",
38             "Is Fabric Exclusive for Customer": "No"

```

```

39     },
40     {
41         "rowId": "d6793b02-8410-464c-9d7a-686d395c3850",
42         "rowIndex": 4,
43         "Mill Article#": "def",
44         "Coloration Process": "Fiber Heather) Dyed"
45     },
46     {
47         "rowId": "13808548-000b-45fa-8574-75d5b79f3ddf",
48         "rowIndex": 6,
49         "Mill Article#": "zxc",
50         "Is Fabric Exclusive for Customer": "Yes",
51         "Coloration Process": "Print"
52     }
53 ],
54 "createdByUserId": "670e275c2ac1b0026b5e55a9",
55 "createdByUsername": "Tommy Wong",
56 "createdDate": "2025-01-08T02:51:18.171Z",
57 "updatedByUserId": "670e275c2ac1b0026b5e55a9",
58 "updatedByUsername": "Tommy Wong",
59 "updatedAt": "2025-01-08T02:51:18.171Z"
60 }

```

Documents

POST

Create Document

POST /v1/documents - **create document record and fields**

- mandatory checking for `documentName` and `workspaceId`
- validation of `documentName` and `workspaceId`
- `maximumLines`: maximum lines allowed

e.g. /v1/documents

Request body:

```

1  {
2      "documentName" : "TW Testing 20241121",
3      "workspaceId" : "66d7c3fc61bcd115bd56bf2f",
4      "maximumLines": 1000
5  }

```

Response code: 201

Response body:

```

1  {
2      "data": {
3          "documentId": "677794b0d5bc7818309e6dfe"
4      }
5  }

```

PUT

Update Document

PUT /v1/documents/{documentId} - **update document and fields**

- mandatory checking for `fieldName`, `type`

e.g. /v1/documents/66d016d10a60b06ee18dd048

Request body:

```
1 {
2   "dynamicFields": [
3     {
4       "fieldName": "supplier code",
5       "fieldDescription": "",
6       "type": "string"
7     },
8     {
9       "fieldName": "Quantity",
10      "fieldDescription": "",
11      "type": "number",
12      "format": {
13        "output": "percent",
14        "decimalPlaces": 3,
15        "thousandSeparated": "Y"
16      }
17    },
18    {
19      "fieldName": "Price",
20      "fieldDescription": "",
21      "type": "number"
22    },
23    {
24      "fieldName": "Total",
25      "fieldDescription": "",
26      "type": "number",
27      "format": {},
28      "formula": "{Quantity}*{Price}"
29    },
30    {
31      "fieldName": "checkbox",
32      "type": "checkbox"
33    },
34    {
35      "fieldName": "image",
36      "fieldDescription": "",
37      "type": "contact"
38    },
39    {
40      "fieldName": "gender",
41      "fieldDescription": "",
42      "type": "array",
43      "values": [ "boy", "girl", "n.a." ]
44    },
45    {
46      "fieldName": "datefield",
47      "fieldDescription": "",
48      "type": "date",
49      "dateFormat": "DD-MMM"
50    },
51    {
52      "fieldName": "lookupfield",
53      "fieldDescription": "",
```

```

54         "type": "lookup",
55         "dataSource": {
56             "requestId": "671f61c586bc75ff510adff7",
57             "lookupViewId": "671f610486bc75ff510adf3e",
58             "lookupTargetFieldName": "Supplier Full Name",
59             "sourceFieldName": "supplier code"
60         }
61     }
62 ]
63 }

```

Response code: 204

PATCH

Update Document Name

PATCH /v1/documents/{documentId} - **update document name**

- mandatory checking for `documentName`
- `maximumLines`: maximum lines allowed

e.g. /v1/documents/66d016d10a60b06ee18dd048 - **update document name**

Request body:

```

1 {
2   "documentName" : "TW TESTING COLLECTION",
3   "maximumLines" : 1000
4 }

```

Response code: 204

GET

Get Document

GET /v1/documents/{documentId} - **get a document record**

e.g. /v1/documents/66f261583f030e34ce0a9039

Get Documents

GET /v1/documents - **search and list document record(s) with criteria**

mandatory checking for `workspacename`

Request parameter:

`workspacename={string}` (whole word match)

`documentname={string}` (partial word match)

e.g. /v1/documents?workspacename=workspace&documentname=TW

Response code: 200

Response body:

```

1 {
2   "data": [

```

```

3      {
4          "documentId": "6787597e2011eb0f2c3c6fa0",
5          "documentName": "Desmond Test - Document 3",
6          "fieldCount": 0,
7          "fieldId": "6787597e2011eb0f2c3c6fa1",
8          "fields": [],
9          "statusCount": 0,
10         "viewCount": 0,
11         "workspaceId": "66d7c3fc61bcd115bd56bf2f",
12         "createdByUserId": "673c63e42ac1b0026b5e55bb",
13         "createdByUserName": "Desmond Chan",
14         "createdDate": "2025-01-15T06:45:18.062Z",
15         "updatedByUserId": "673c63e42ac1b0026b5e55bb",
16         "updatedByUserName": "Desmond Chan",
17         "updatedAt": "2025-01-15T06:45:18.062Z"
18     },
19     {
20         "documentId": "673d472590b6d462a12264be",
21         "documentName": "Doc - Desmond",
22         "fieldCount": 4,
23         "fieldId": "673d472590b6d462a12264bf",
24         "fields": [
25             {
26                 "fieldId": "673d53d290b6d462a1226698",
27                 "fieldName": "Name",
28                 "type": "string"
29             },
30             {
31                 "fieldId": "673d53d290b6d462a1226699",
32                 "fieldName": "Age",
33                 "type": "number",
34                 "format": {},
35                 "formatString": ""
36             },
37             {
38                 "fieldId": "673d53d290b6d462a122669a",
39                 "fieldName": "Gender",
40                 "type": "array",
41                 "values": [
42                     "M",
43                     "W"
44                 ]
45             },
46             {
47                 "fieldId": "67ad48609965d73bc5cf6ed0",
48                 "fieldName": "photo",
49                 "type": "contact"
50             }
51         ],
52         "statusCount": 1,
53         "viewCount": 1,
54         "workspaceId": "66d7c3fc61bcd115bd56bf2f",
55         "createdByUserId": "673c63e42ac1b0026b5e55bb",
56         "createdByUserName": "Desmond Chan",
57         "createdDate": "2024-11-20T02:19:17.432Z",
58         "updatedByUserId": "673c63e42ac1b0026b5e55bb",
59         "updatedByUserName": "Desmond Chan",
60         "updatedAt": "2024-11-20T03:31:27.170Z"
61     },
62     {
63         "documentId": "673ff88da3d011aba35eba2d",
64         "documentName": "Documents - Desmond",
65         "fieldCount": 9,
66         "fieldId": "673ff88da3d011aba35eba2e",
67         "fields": [
68             {
69                 "fieldId": "673ffbdfa3d011aba35eba2f",

```

```

70         "fieldName": "Feature Code",
71         "type": "string"
72     },
73     {
74         "fieldId": "673ffbdfa3d011aba35eba30",
75         "fieldName": "Season",
76         "type": "string"
77     },
78     {
79         "fieldId": "673ffbdfa3d011aba35eba31",
80         "fieldName": "Quantity",
81         "type": "number",
82         "format": {},
83         "formatString": ""
84     },
85     {
86         "fieldId": "673ffbdfa3d011aba35eba32",
87         "fieldName": "Price",
88         "type": "number",
89         "format": {
90             "decimalPlaces": 2,
91             "output": "number",
92             "thousandSeparated": "Y"
93         },
94         "formatString": "#, #0.00"
95     },
96     {
97         "fieldId": "673ffbdfa3d011aba35eba33",
98         "fieldName": "Total Amount",
99         "type": "number",
100        "format": {},
101        "formatString": "",
102        "formula": "{673ffbdfa3d011aba35eba31}*
{673ffbdfa3d011aba35eba32}",
103        "formulaParticipateColumnId": [
104            {
105                "id": "673ffbdfa3d011aba35eba31"
106            },
107            {
108                "id": "673ffbdfa3d011aba35eba32"
109            }
110        ]
111    },
112    {
113        "fieldId": "67a05ee044789378196d1493",
114        "fieldName": "create date",
115        "type": "date",
116        "dateFormat": "YYYY/MM/DD"
117    },
118    {
119        "fieldId": "67935cc74315a6d63eb67621",
120        "fieldName": "photo",
121        "type": "contact"
122    },
123    {
124        "fieldId": "67935cc74315a6d63eb67622",
125        "fieldName": "checkbox",
126        "type": "checkbox"
127    },
128    {
129        "fieldId": "67a576fc860c8ad733a3c65b",
130        "fieldName": "gender",
131        "type": "array",
132        "align": "left",
133        "values": [
134            "Men",
135            "Women"

```

```

136         ]
137     }
138 ],
139     "statusCount": 3,
140     "viewCount": 3,
141     "workspaceId": "66d7c3fc61bcd115bd56bf2f",
142     "createdByUserId": "673c63e42ac1b0026b5e55bb",
143     "createdByUserName": "Desmond Chan",
144     "createdDate": "2024-11-22T03:20:45.589Z",
145     "updatedByUserId": "673c63e42ac1b0026b5e55bb",
146     "updatedByUserName": "Desmond Chan",
147     "updatedAt": "2025-02-03T06:02:47.068Z"
148 },
149 {
150     "documentId": "67777e076aaa73472a9045ed",
151     "documentName": "Documents - Desmond 2",
152     "fieldCount": 5,
153     "fieldId": "67777e086aaa73472a9045ee",
154     "fields": [
155         {
156             "fieldId": "677780d36aaa73472a9045f4",
157             "fieldName": "Image",
158             "type": "contact"
159         },
160         {
161             "fieldId": "677780d36aaa73472a9045f5",
162             "fieldName": "Name",
163             "type": "string"
164         },
165         {
166             "fieldId": "677780d36aaa73472a9045f6",
167             "fieldName": "Gender",
168             "type": "array",
169             "values": [
170                 "M",
171                 "W",
172                 "NA"
173             ]
174         },
175         {
176             "fieldId": "677780d36aaa73472a9045f7",
177             "fieldName": "Height",
178             "type": "number",
179             "format": {},
180             "formatString": ""
181         },
182         {
183             "fieldId": "6784bfcd1e9388407bec3204",
184             "fieldName": "Image2",
185             "type": "contact"
186         }
187     ],
188     "statusCount": 3,
189     "viewCount": 2,
190     "workspaceId": "66d7c3fc61bcd115bd56bf2f",
191     "createdByUserId": "673c63e42ac1b0026b5e55bb",
192     "createdByUserName": "Desmond Chan",
193     "createdDate": "2025-01-03T06:04:55.697Z",
194     "updatedByUserId": "673c63e42ac1b0026b5e55bb",
195     "updatedByUserName": "Desmond Chan",
196     "updatedAt": "2025-01-08T02:39:18.882Z"
197 },
198 {
199     "documentId": "677b4edd6ca507b825106af6",
200     "documentName": "Documents - Desmond 3",
201     "fieldCount": 0,
202     "fieldId": "677b4edd6ca507b825106af7",

```



```

203     "fields": [],
204     "statusCount": 0,
205     "viewCount": 0,
206     "workspaceId": "66d7c3fc61bcd115bd56bf2f",
207     "createdByUserId": "673c63e42ac1b0026b5e55bb",
208     "createdByUserName": "Desmond Chan",
209     "createdDate": "2025-01-06T03:32:45.031Z",
210     "updatedByUserId": "673c63e42ac1b0026b5e55bb",
211     "updatedByUserName": "Desmond Chan",
212     "updatedAt": "2025-01-06T03:32:45.031Z"
213 },
214 {
215     "documentId": "6746807d6d9bb8104c1bc5f5",
216     "documentName": "doc2 - Desmond",
217     "fieldCount": 2,
218     "fieldId": "6746807d6d9bb8104c1bc5f6",
219     "fields": [
220         {
221             "fieldName": "name",
222             "type": "string",
223             "fieldId": "674680966d9bb8104c1bc5f7"
224         },
225         {
226             "fieldName": "age",
227             "type": "number",
228             "format": {},
229             "formatString": "",
230             "fieldId": "674680966d9bb8104c1bc5f8"
231         }
232     ],
233     "statusCount": 1,
234     "viewCount": 1,
235     "workspaceId": "66d7c3fc61bcd115bd56bf2f",
236     "createdByUserId": "673c63e42ac1b0026b5e55bb",
237     "createdByUserName": "Desmond Chan",
238     "createdDate": "2024-11-27T02:14:21.231Z",
239     "updatedByUserId": "673c63e42ac1b0026b5e55bb",
240     "updatedByUserName": "Desmond Chan",
241     "updatedAt": "2024-11-27T02:14:21.231Z"
242 },
243 {
244     "documentId": "674681466d9bb8104c1bc5fe",
245     "documentName": "doc3 - desmond",
246     "fieldCount": 2,
247     "fieldId": "674681466d9bb8104c1bc5ff",
248     "fields": [
249         {
250             "fieldName": "name",
251             "type": "string",
252             "fieldId": "674681606d9bb8104c1bc600"
253         },
254         {
255             "fieldName": "age",
256             "type": "number",
257             "format": {},
258             "formatString": "",
259             "fieldId": "674681606d9bb8104c1bc601"
260         }
261     ],
262     "statusCount": 1,
263     "viewCount": 1,
264     "workspaceId": "66d7c3fc61bcd115bd56bf2f",
265     "createdByUserId": "673c63e42ac1b0026b5e55bb",
266     "createdByUserName": "Desmond Chan",
267     "createdDate": "2024-11-27T02:17:42.291Z",
268     "updatedByUserId": "673c63e42ac1b0026b5e55bb",
269     "updatedByUserName": "Desmond Chan",

```

```

270     "updatedAt": "2024-11-27T02:17:42.291Z"
271   }
272 ]
273 }

```

Views

POST

Create View

POST /v1/views - create a view

- mandatory checking for `viewName`, `documentId`, `workspaceId`, `order`
- validation of `order`, `fieldName`, `mandatory`, `readOnly`

e.g. /v1/views

Request body:

```

1  {
2    "viewName" : "DC View Name 8",
3    "allowInsertRow": "Y",
4    "allowRemoveRow": "Y",
5    "documentId" : "67b694965294d4f277d34ce6",
6    "workspaceId" : "66d7c3fc61bcd115bd56bf2f",
7    "order": 8,
8    "dynamicFields": [
9      {
10       "fieldName": "product desc",
11       "mandatory": "N",
12       "readOnly": "N"
13     },
14     {
15       "fieldName": "Quantity",
16       "mandatory": "N",
17       "readOnly": "N"
18     },
19     {
20       "fieldName": "Price",
21       "readOnly": "N"
22     },
23     {
24       "fieldName": "Total",
25       "mandatory": "N",
26       "readOnly": "N"
27     },
28     {
29       "fieldName": "checkbox",
30       "mandatory": "N",
31       "readOnly": "Y"
32     },
33     {
34       "fieldName": "image",
35       "mandatory": "Y"
36     },
37     {
38       "fieldName": "gender",
39       "mandatory": "N",
40       "readOnly": "N"
41     },
42     {
43       "fieldName": "datefield",

```

```

44     "mandatory": "N",
45     "readOnly": "N"
46   },
47   {
48     "fieldName": "lookupfield",
49     "mandatory": "N",
50     "readOnly": "N"
51   },
52   {
53     "fieldName": "a",
54     "mandatory": "N",
55     "readOnly": "N"
56   },
57   {
58     "fieldName": "b",
59     "mandatory": "N",
60     "readOnly": "N"
61   },
62   {
63     "fieldName": "c",
64     "mandatory": "N",
65     "readOnly": "N"
66   }
67 ]
68 }

```

Response code: 201

Response body:

```

1  {
2    "data": {
3      "viewId": "67bea10463a9838e14bbee21"
4    }
5  }

```

PUT

Update View

PUT /v1/views/{viewId} - **update view record**

- mandatory checking for `viewName`, `order`, `{viewId}`
- checking - the field can either `mandatory` or `readOnly`

e.g. /v1/views/67bd5c7a0bab33a7f0bdf0e9

Request body:

```

1  {
2    "viewName" : "DC View Name 8b",
3    "order": 8,
4    "allowInsertRow": "Y",
5    "allowRemoveRow": "Y",
6    "dynamicFields": [
7      {
8        "fieldName": "product desc",
9        "mandatory": "Y",
10       "readOnly": "N"
11     },
12     {
13       "fieldName": "Quantity",
14       "mandatory": "N",

```

```
15     "readOnly": "Y"
16   }
17 ]
18 }
```

Response code: 204

GET

Get View

GET /v1/views/{viewId} - **get a single view record**

e.g. /v1/views/66f261583f030e34ce0a9039

Get Views

GET /v1/views - **search and list record(s) with criteria**

mandatory checking for `workspacename`

Request parameter:

`workspacename={string}` (whole word match)

`documentname={string}` (whole word match)

`viewname={string}` (partial word match)

e.g. /v1/views?viewname=desmond&workspacename=workspace&documentname=DC Doc
20250226

Response code: 200

Response body:

```
1  {
2    "data": [
3      {
4        "viewId": "66f62c2105862e2e75b15d4c",
5        "viewName": "TW Testing View 20241015",
6        "order": 1,
7        "documentId": "66f261583f030e34ce0a9039",
8        "documentName": "TW Testing Doc Type 2",
9        "dynamicFields": [
10         {
11           "fieldId": "66f2619d3f030e34ce0a903b",
12           "fieldName": "Total Price",
13           "type": "number",
14           "format": {
15             "decimalPlaces": 0,
16             "output": "number",
17             "thousandSeparated": "N"
18           },
19           "formatString": "",
20           "isSelected": true,
21           "values": []
22         },
23         {
24           "fieldId": "66f62bfe05862e2e75b15d4b",
25           "fieldName": "Price",
26           "type": "number",
27           "format": {
28             "decimalPlaces": 0,
```

```

29         "output": "number",
30         "thousandSeparated": "N"
31     },
32     "formatString": "",
33     "isSelected": true
34 }
35 ],
36 "createdByUserId": "670e275c2ac1b0026b5e55a9",
37 "createdByUserName": "Tommy Wong",
38 "createdDate": "2025-01-08T02:51:18.171Z",
39 "updatedByUserId": "670e275c2ac1b0026b5e55a9",
40 "updatedByUserName": "Tommy Wong",
41 "updatedAt": "2025-01-08T02:51:18.171Z"
42 },
43 {
44     "viewId": "66f62c7305862e2e75b15d4f",
45     "viewName": "TW Testing View 2",
46     "order": 2,
47     "documentId": "66f261583f030e34ce0a9039",
48     "documentName": "TW Testing Doc Type 2",
49     "dynamicFields": [
50         {
51             "fieldId": "66f2619d3f030e34ce0a903b",
52             "fieldName": "Total Price",
53             "type": "number",
54             "format": {
55                 "decimalPlaces": 0,
56                 "output": "number",
57                 "thousandSeparated": "N"
58             },
59             "formatString": "",
60             "isSelected": true,
61             "values": []
62         }
63     ],
64     "createdByUserId": "670e275c2ac1b0026b5e55a9",
65     "createdByUserName": "Tommy Wong",
66     "createdDate": "2025-01-08T02:51:18.171Z",
67     "updatedByUserId": "670e275c2ac1b0026b5e55a9",
68     "updatedByUserName": "Tommy Wong",
69     "updatedAt": "2025-01-08T02:51:18.171Z"
70 }
71 ]
72 }

```

Statuses

POST

Create Status

POST /v1/statuses - create status record for request

- mandatory checking for name
- validation of document and workspace

e.g. /v1/statuses

Request body:

```

1 {
2     "statusName": "Done",
3     "documentId": "66f261583f030e34ce0a9039",

```

```

4     "order": 3,
5     "defaultViewId": "66f62c2105862e2e75b15d4c",
6     "statusCategoryId": "670dd41dd3bc14b6e8c284ae",
7     "workspaceId" : "66d7c3fc61bcd115bd56bf2f",
8     "views": [
9         {
10            "viewId": "66f62c2105862e2e75b15d4c"
11        },
12        {
13            "viewId": "66f62c7305862e2e75b15d4f"
14        }
15    ]
16 }

```

Response code: 201

Response body:

```

1  {
2      "data": {
3          "statusId": "677e0fdad074239887e31ee2"
4      }
5  }

```

PUT

Update Status

PUT /v1/statuses/{statusId} - *update a status record for a request*

e.g. /v1/statuses/677e0fdad074239887e31ee2

Request body:

```

1  {
2      "statusName": "Done",
3      "order": 3,
4      "defaultViewId": "66f62c2105862e2e75b15d4c",
5      "statusCategoryId": "670dd41dd3bc14b6e8c284ae",
6      "views": [
7          {
8              viewId: "66f62c2105862e2e75b15d4c"
9          },
10         {
11             viewId: "66f62c7305862e2e75b15d4f"
12         }
13     ]
14 }

```

Response code: 204

GET

Get Status

GET /v1/statuses - *get a single status record*

e.g. /v1/statuses/66f261583f030e34ce0a9039

Get Statuses

GET /v1/statuses - **search and list status record(s) with criteria**

Request parameter:

workspacename={string} (whole word match)

documentname={string} (whole word match)

statusname={string} (partial word match)

statuscategoryname={string} (partial word match)

page={number}

pagesize={number}

keyword={string}

e.g. /v1/statuses?statusname=Done&workspacename=workspace&documentname=DC Doc
20250226

Response code: 200

Response body:

```
1 {
2   "data": [
3     {
4       "statusId": "677e0fdad074239887e31ee2",
5       "statusName": "Done",
6       "documentId": "66f261583f030e34ce0a9039",
7       "documentName": "TW Testing Doc Type 2",
8       "order": 3,
9       "statusCategory": "670dd41dd3bc14b6e8c284ae",
10      "statusCategoryData": {
11        "id": "670dd41dd3bc14b6e8c284ae",
12        "name": "Done",
13        "backgroundColor": "#DCFFF1",
14        "textColor": "#576E4E"
15      },
16      "defaultViewId": "66f62c2105862e2e75b15d4c",
17      "defaultViewName": "TW Testing View 20241015",
18      "views": [
19        {
20          "viewId": "66f62c2105862e2e75b15d4c"
21        },
22        {
23          "viewId": "66f62c7305862e2e75b15d4f"
24        }
25      ],
26      "createdByUserId": "670e275c2ac1b0026b5e55a9",
27      "createdByUserName": "Tommy Wong",
28      "createdDate": "2025-01-08T02:51:18.171Z",
29      "updatedByUserId": "670e275c2ac1b0026b5e55a9",
30      "updatedByUserName": "Tommy Wong",
31      "updatedAt": "2025-01-08T02:51:18.171Z"
32    },
33    {
34      "statusId": "66c853059b23805ef19e1de7",
35      "statusName": "DONE",
36      "order": 2,
37      "documentId": "66c84c639b23805ef19e1dc8",
38      "documentName": "LE WIP",
39      "statusCategory": "670dd41dd3bc14b6e8c284ae",
```

```

40         "statusCategoryData": {
41             "id": "670dd41dd3bc14b6e8c284ae",
42             "name": "Done",
43             "backgroundColor": "#DCFFF1",
44             "textColor": "#576E4E"
45         },
46         "defaultViewId": "66c852d69b23805ef19e1de5",
47         "defaultViewName": "Default",
48         "views": null,
49         "createdByUserId": "670e275c2ac1b0026b5e55a9",
50         "createdByUserName": "Tommy Wong",
51         "createdDate": "2025-01-08T02:51:18.171Z",
52         "updatedByUserId": "670e275c2ac1b0026b5e55a9",
53         "updatedByUserName": "Tommy Wong",
54         "updatedAt": "2025-01-08T02:51:18.171Z"
55     },
56 ]
57 }

```

Workspaces

POST

Create workspaces (next phase)

POST /v1/workspaces - **create workspace**

- mandatory checking for workspaceName
- duplicate name checking

e.g. /v1/workspaces

Request body:

```

1 {
2     "workspaceName": "New Workspace"
3 }

```

Response code: 201

Response body:

```

1 {
2     "data": {
3         "workspaceId": "677e0fdad074239887e31ee2"
4     }
5 }

```

PUT

Update Workspaces (next phase)

PUT /v1/workspace/{workspaceId} - **update workspace name**

e.g. /v1/statuses/677e0fdad074239887e31ee2

Request body:

```

1 {
2     "workspaceName": "New Workspace"

```



```
3 }
```

Response code: 204

GET

Get Workplace

GET /v1/workspaces - **get a single workspace record**

e.g. /v1/workspaces/66d7c3fc61bcd115bd56bf2f

Response code: 200

Response body:

```
1 {
2   "data": [
3     {
4       "workspaceName": "workspace"
5     }
6   ]
7 }
```

POST

Add Workspace User

POST /v1/workspaces/{workspaceId}/users/{email} - **add a user to workspace**

- mandatory checking for {workspaceId} and {email}

e.g. /v1/workspaces/66d7c3fc61bcd115bd56bf2f/users/tommywong@lifung.com

Response code: 201

PUT

Update Workspace User's authorizations

PUT /v1/workspaces/{workspaceId}/users/{email}/authorizations - **add a user to workspace**

- mandatory checking for {workspaceId} and {email}
- user authorization checking
- validation

e.g. /v1/workspaces/66d7c3fc61bcd115bd56bf2f/users/tommywong@lifung.com/authorizations

Request body:

```
1 {
2   "admin": "Y",
3   "documentSetupBasic": "Y",
4   "documentSetupAdvanced": "Y",
5   "requestCreate": "Y",
```

```

6   "requestUpdate": "Y",
7   "requestRead": "Y",
8   "requestShare": "Y",
9   "requestLink": "Y",
10  "requestChangeStatus": "Y",
11  "requestDataUpdate": "Y",
12  "collectionCreate": "Y",
13  "collectionUpdate": "N",
14  "collectionRead": "Y"
15 }

```

Response code: 204

GET

Get Workspace User

GET /v1/workspaces/{workspaceId}/users/{userEmail} - **get users of a workspace by email**

e.g. /v1/workspaces/66d7c3fc61bcd115bd56bf2f/users/tommywong@lifung.com

GET /v1/workspaces/{workspaceId}/users/{userId} - **get users of a workspace by user id**

e.g. /v1/workspaces/66d7c3fc61bcd115bd56bf2f/users/670e275c2ac1b0026b5e55a9

Get Workspace Users

GET /v1/workspaces/{workspaceId}/users - **get all users of a workspace**

Request parameter:

username={string} (partial word match & case insensitive)

- user authorization - workspace admin

e.g. /v1/workspaces/66d7c3fc61bcd115bd56bf2f/users?username=tommy

Response code: 200

Response body:

```

1  {
2    "data": [
3      {
4        "userId": "670e275c2ac1b0026b5e55a9",
5        "userName": "Tommy Wong",
6        "userEmail": "tommywong@lifung.com",
7        "authorization": {
8          "admin": "Y",
9          "documentSetupBasic": "Y",
10         "documentSetupAdvanced": "Y",
11         "requestCreate": "Y",
12         "requestUpdate": "Y",
13         "requestRead": "Y",
14         "requestShare": "Y",
15         "requestChangeStatus": "Y",
16         "requestDataUpdate": "Y",
17         "collectionCreate": "Y",
18         "collectionUpdate": "N",
19         "collectionRead": "Y"
20       }
21     },

```

```

22     {
23         "userId": "670e275c2ac1b0026b5e55a9",
24         "userName": "Tommy Wong",
25         "userEmail": "tommywong@lifung.com"
26         "authorization": {
27             "admin": "Y",
28             "documentSetupBasic": "Y",
29             "documentSetupAdvanced": "Y",
30             "requestCreate": "Y",
31             "requestUpdate": "Y",
32             "requestRead": "Y",
33             "requestShare": "Y",
34             "requestChangStatus": "Y",
35             "requestDataUpdate": "Y",
36             "collectionCreate": "Y",
37             "collectionUpdate": "N",
38             "collectionRead": "Y",
39             "permissionId": "670e29092ac1b0026b5e55ab",
40             "permission": {
41                 "resources": [
42                     "setup:documentType",
43                     "setup:masteField",
44                     "setup:view",
45                     "setup:status",
46                     "request",
47                     "collection",
48                     "document",
49                     "user"
50                 ],
51                 "request:create": "Y",
52                 "request:read": "Y",
53                 "request:update": "Y",
54                 "request:document:read": "Y",
55                 "request:document:createUpdate": "Y",
56                 "request:access:read": "Y",
57                 "request:access:update": "Y",
58                 "request:status:update": "Y",
59                 "request:assign:update": "Y",
60                 "collection:create": "Y",
61                 "collection:read": "Y",
62                 "collection:update": "Y",
63                 "request:collection:read": "Y",
64                 "request:delete": "Y",
65                 "request:document:delete": "Y",
66                 "collection:delete": "Y",
67                 "user:workspaceAdmin": "Y",
68                 "document": "Y",
69                 "document.all": "Y"
70             }
71         }
72     }
73 ]
74 }

```

StatusCategories

GET

Get Status Categories

GET /v1/statuscategories - *list all records of status categories*

e.g. /v1/statuscategories

Response code: 200

Response body:

```
1 {
2   "data": [
3     {
4       "statusCategoryId": "670dc553d3bc14b6e8c284ac",
5       "statusCategoryName": "To Do",
6       "textColor": "#172B4D",
7       "backgroundColor": "#F1F2F4",
8       "isDefault": "Y",
9       "isCompleted": "N",
10      "order": 1
11    },
12    {
13      "statusCategoryId": "670dd410d3bc14b6e8c284ad",
14      "statusCategoryName": "In Progress",
15      "textColor": "#4B77CF",
16      "backgroundColor": "#E9F2FF",
17      "isDefault": "N",
18      "isCompleted": "N",
19      "order": 2
20    },
21    {
22      "statusCategoryId": "670dd41dd3bc14b6e8c284ae",
23      "statusCategoryName": "To Do",
24      "textColor": "#576E4E",
25      "backgroundColor": "#DCFF11",
26      "isDefault": "N",
27      "isCompleted": "Y",
28      "order": 3
29    }
30  ]
31 }
```

Users

POST

Create User

POST /v1/users - **create a user**

- mandatory checking for email and userName
- validation of email
- administrator (Y/N) will be "N" by default if not specified
- activeUser (Y/N) will be "Y" by default if not specified
- user authorization checking of user creation

e.g. /v1/users

Request body:

```
1 {
2   "email" : "tommywong@lifung.com",
3   "userName" : "Tommy Wong",
4   "partnerCompany" : "LF",
5   "referrerEmail" : "",
```

```

6     "administrator" : "Y",
7     "activeUser" : "Y",
8     "internalRemark" : "Testing"
9 }

```

Response code: 201

Response body:

```

1 {
2   "data": {
3     "userId": "677794b0d5bc7818309e6dfe"
4   }
5 }

```

PUT

Update User (next phase)

PUT /v1/users - **update a user**

- mandatory checking for email, userName
- administrator (Y/N) will be "N" by default if not specified
- activeUser (Y/N) will be "Y" by default if not specified
- validation of email
- user authorization checking of user creation

e.g. /v1/users

Request body:

```

1 {
2   "email" : "tommywong@lifung.com",
3   "userName" : "Tommy Wong",
4   "partnerCompany" : "LF",
5   "referrerEmail" : "",
6   "isAdmin" : "Y",
7   "isActive" : "Y",
8   "internalRemark" : "Testing"
9 }

```

Response code: 204

Response body:

```

1 {
2   "data": {
3     "userId": "677794b0d5bc7818309e6dfe"
4   }
5 }

```

GET

Get User

GET /v1/users/{userEmail} - **get user by email**

e.g. /v1/users/tommywong@lifung.com

GET /v1/users/{userId} - **get user by id**

e.g. /v1/users/66f65d6a2ac1b0026b5e559d

Get Users

GET /v1/users - **get all users and filtered by parameters**

Request parameter:

workspaceid={string}

username={string} (partial word match & case insensitive)

e.g. /v1/users?workspaceid=66d7c3fc61bcd115bd56bf2f

e.g. /v1/users?workspaceid=66d7c3fc61bcd115bd56bf2f&username=tommy

e.g. /v1/users?username=tommy

- user authorization (application admin or workspace admin)

Response code: 200

Response body:

```
1 {
2   "data": [
3     {
4       "userId": "670e275c2ac1b0026b5e55a9",
5       "userName": "Tommy Wong (Supplier)",
6       "userEmail": "smaapp.sup3@mail.com",
7       "partnerCompany": "LF",
8       "referrerEmail": "tommywong@lifung.com",
9       "isAdmin": "N",
10      "isActive": "Y",
11      "internalRemark": "Internal Remarks line 1\nInternal Remarks
line 2\nInternal Remarks line 3",
12    },
13    {
14      "userId": "670e275c2ac1b0026b5e55a9",
15      "userName": "Tommy Wong",
16      "userEmail": "tommywong@lifung.com",
17      "isAdmin": "Y",
18      "isActive": "Y",
19      "internalRemark": "Internal Remarks line 1\nInternal Remarks
line 2\nInternal Remarks line 3",
20    }
21  ]
22 }
```

less data if shown as below if user is a workspace admin only

```
1 {
2   "data": [
3     {
4       "userId": "670e275c2ac1b0026b5e55a9",
5       "userName": "Tommy Wong (Supplier)",
6       "userEmail": "smaapp.sup3@mail.com",
7       "isActive": "N"
8     }
9   ]
10 }
```

```

8      },
9      {
10         "userId": "670e275c2ac1b0026b5e55a9",
11         "userName": "Tommy Wong",
12         "userEmail": "tommywong@lifung.com"
13         "isActive": "Y"
14     }
15 ]
16 }

```

Get Me

GET /v1/users/me - **get current user info**

e.g. /v1/users/me

Response code: 200

Response body:

```

1  {
2      "data": {
3          "userId": "670e275c2ac1b0026b5e55a9",
4          "userName": "Tommy Wong",
5          "userType": "LF",
6          "userEmail": "tommywong@lifung.com",
7          "isAdmin": "Y",
8          "timeZone": "Asia/Bangkok",
9          "timeZoneReference": "(GMT+7:00) Bangkok, Hanoi, Jakarta",
10         "inputDateFormat": "YYYY-MM-DD",
11         "defaultWorkspaceId": "66d7c3fc61bcd115bd56bf2f",
12         "workspaces": [
13             {
14                 "workspaceId": "66d7c3fc61bcd115bd56bf2f",
15                 "workspaceName": "workspace"
16             },
17             {
18                 "workspaceId": "66d8114e93ecbe0f6a85f5e0",
19                 "workspaceName": "workspace 2"
20             }
21         ],
22         "createdByUserId": "670e275c2ac1b0026b5e55a9",
23         "createdByUserName": "Tommy Wong",
24         "createdDate": "2025-01-08T02:51:18.171Z",
25         "updatedByUserId": "670e275c2ac1b0026b5e55a9",
26         "updatedByUserName": "Tommy Wong",
27         "updatedAt": "2025-01-08T02:51:18.171Z"
28     }
29 }

```

UserGroups

POST

Create User Group

POST /v1/usergroups - **create a user group record**

- mandatory checking for `userGroupName` , `workspaceId`
- validation of `userId` and `workspaceId`

e.g. /v1/usergroups

Request body:

```
1 {  
2   "userGroupName" : "Testing Group",  
3   "workspaceId" : "66d7c3fc61bcd115bd56bf2f",  
4   "users": [  
5     {  
6       "userId": "677794b0d5bc7818309e6dfe"  
7     }  
8   ]  
9 }
```

Response code: 201

Response body:

```
1 {  
2   "data": {  
3     "userGroupId": "677de826c491e9c07833f862"  
4   }  
5 }
```

PUT

Update User Group

PUT /v1/usergroups/{userGroupId} - **update user group record**

- mandatory checking for userGroupName
- validation of requestId

e.g. /v1/usergroups/677de826c491e9c07833f862

Request body:

```
1 {  
2   "userGroupName" : "Testing Group",  
3   "users": [  
4     {  
5       "userId": "677794b0d5bc7818309e6dfe"  
6     }  
7   ]  
8 }
```

Response code: 204

GET

Get User Group

GET /v1/usergroups/{userGroupId} - **get a single user group record**

e.g. /v1/usergroups/677de826c491e9c07833f862

Get User Groups

GET /v1/usergroups - **search and list record(s) with criteria**

Request parameter:

workspacename={string} (whole word match)

usergroupname={string} (partial word match)

e.g. /v1/usergroups?workspacename=workspace&usergroupname=TESTING%20Group

Response code: 200

Response body:

```
1 {
2   "data": [
3     {
4       "userGroupId": "677de826c491e9c07833f862",
5       "userGroupName": "TESTING Group",
6       "workspaceId": "66d7c3fc61bcd115bd56bf2f",
7       "users": [
8         {
9           "userId": "677794b0d5bc7818309e6dfe"
10        },
11        {
12          "userId": "677794b0d5bc781dffdfdfdf"
13        }
14      ],
15      "createdByUserId": "670e275c2ac1b0026b5e55a9",
16      "createdByUserName": "Tommy Wong",
17      "createdDate": "2025-01-08T02:51:18.171Z",
18      "updatedByUserId": "670e275c2ac1b0026b5e55a9",
19      "updatedByUserName": "Tommy Wong",
20      "updatedAt": "2025-01-08T02:51:18.171Z"
21    },
22    {
23      "userGroupId": "677de826c491e9c07833sdsds",
24      "userGroupName": "TESTING Group 2",
25      "workspaceId": "66d7c3fc61bcd115bd56bf2f",
26      "users": [
27        {
28          "userId": "677794b0d5bc78dfdfde6dfe",
29        },
30        {
31          "userId": "677794b0d5bc78dssdadfd"
32        }
33      ],
34      "createdByUserId": "670e275c2ac1b0026b5e55a9",
35      "createdByUserName": "Tommy Wong",
36      "createdDate": "2025-01-08T02:51:18.171Z",
37      "updatedByUserId": "670e275c2ac1b0026b5e55a9",
38      "updatedByUserName": "Tommy Wong",
39      "updatedAt": "2025-01-08T02:51:18.171Z"
40    }
41  ]
42 }
```

Date Time Format

Timestamp

[ISO 8601](#)

UTC - Coordinated Universal Time

YYYY-MM-DDThh:mm:ss.sssZ

e.g. 2023-07-30T18:51:49.196Z

Status Code

- 1xx for information
- 2xx for success
 - 200 OK
 - 201 Created
 - 204 No Content
- 3xx for redirection
- 4xx for client errors
 - 400 Bad Request
 - 401 Unauthorized
 - 403 Forbidden
 - 404 Not Found
 - 405 Method Not Allowed
 - 422 Unprocessable Content
- 5xx for server errors

reference: [🏠 RFC 9110: HTTP Semantics | RFC Editor](#)